

Global Warming Science

<https://courses.seas.harvard.edu/climate/eli/Courses/EPS101/>

Mountain glaciers:

Please use the template below to answer the workshop questions. “XX” indicates places where you need to complete/write code or add a discussion.

Your name:

```
[ ]: # load libraries, read time series data:
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import cartopy.crs as ccrs
from scipy.stats import linregress
from scipy.signal import savgol_filter # for smoothing time series
from scipy import optimize
import pickle
import copy
import warnings
import pandas as pd
import matplotlib.dates as mdates

# load the data from a pickle file:
with open('./mountain_glaciers_variables.pickle', 'rb') as file:
    while 1:
        try:
            d = pickle.load(file)
        except (EOFError):
            break
        # print information about each extracted variable:
        for key in list(d.keys()):
            print(f"extracting pickled variable: name={key}; size={d[key].shape}")
            #print("type=", type(d[key]))
            globals().update(d)

# finish extracting the dictionary with glacier data, and print one entry for example:
front_variation=copy.deepcopy(front_variation.item())
snow_extent_north_hemisphere=copy.deepcopy(snow_extent_north_hemisphere.item())

# print one entry as an example:
first_glacier_name=list(front_variation.keys())[2]
print(
    f"\n\nAs an example, the third [i=2] glacier name: {first_glacier_name}"
    f", and its data: \nyears={front_variation[first_glacier_name][0]}"
    f"\nfront location (m)={front_variation[first_glacier_name][1]}"
    f"\nlon/lat={front_variation[first_glacier_name][2]}"
)

print(f"\nThere are {len(front_variation):d} glacier records")
```

Explanation of variables

front_variation: a dictionary where the key is the 'glacier name'/'2-letter country code'; e.g. VACAS/AR is in Argentina. the elements of the dictionary are the years of data, glacier length relative to 1960 and the [lon,lat] of the glacier. The example above shows how to extract each of these from the dictionary.

Ts_smooth: annually averaged smooth global mean surface temperature

Ts_smooth_years: the corresponding years

Quelccaya_delta18O1976, Quelccaya_depth1976 and Quelccaya_delta18O2016, Quelccaya_depth2016: two ice core records showing seasonal variation of $\delta^{18}\text{O}$.

Quelccaya_years, Quelccaya_delta18O: a longer combined record to be used as a temperature proxy

snow_extent_north_hemisphere: a dictionary with (time, area) snow area data in the northern hemisphere for each season

1) Glacier length records:

(a) Plot the time evolution of all glacier lengths relative to 1960.

```
[ ]: # plot all glaciers as function of year:
fig=plt.figure(figsize=(6,4),dpi=200)
for glacier_name in front_variation.keys():
    years=XX
    locations=XX
    plt.plot(years,locations/1000,'-',lw=0.5 \
            ,markersize=1.5,label=glacier_name)

plt.xlabel(XX)
plt.xlim(XX)
plt.ylabel("front location relative to 1960 (km)")

plt.tight_layout()
plt.show()
```

(b) Plot the length time series of one glacier from each of the following: Argentina, Peru, New Zealand, United States, Russia, Switzerland, and Tajikistan. Choose the glacier with the record with the largest number of data points for each of these countries.

```
[ ]: # plot a few glaciers from different countries.
# see https://www.nationsonline.org/oneworld/country_code_list.htm
# To find them, print the list of glaciers with the length of record
# (number of data points) of each glacier:
if 0: # change to 1 to see list of glaciers
    for glacier_name in front_variation.keys():
        print(f"{glacier_name} {len(front_variation[glacier_name][0]):d}")

# insert glacier names into following list based on the printed full list above:
names_to_plot=['XX','XX',...]

# plot:
fig=plt.figure(figsize=(6,4),dpi=200)
for glacier_name in names_to_plot:
    years=XX
    locations=XX
    # calculate linear regression line for this glacier:
```

```

fit=linregress(XX)
slope=XX;
intercept=XX;
p=XX;
r2=(fit.rvalue)**2
regression_line=intercept+slope*years
# plot:
hl=plt.plot(years,locations/1000,'-',lw=0.5 \
            ,markersize=1.5,label=glacier_name)
# add the regression line in dash, same color:
color=hl[0].get_color()
plt.plot(years,regression_line/1000,'--',lw=0.5,color=color)

plt.xlabel(XX)
plt.xlim(XX)
plt.ylabel("front location relative to 1960 (km)")
plt.legend(fontsize=6)

plt.tight_layout()
plt.show()

```

(c) Calculate and plot the bin-average over all glaciers of glacier length relative to 1960 as a function of year.

```
[ ]: # calculate the bin-averaged record by average over decadal-bins
```

```

props = dict(boxstyle='round', edgecolor="white", facecolor='white', alpha=1.0)

# first, combine all glacier record into a single one:
years_combined = []; locations_combined=[];
for glacier_name in front_variation.keys():
    years=front_variation[glacier_name][0]
    locations=np.asarray(front_variation[glacier_name][1])
    years_combined.extend(years)
    locations_combined.extend(locations)

# bin-averaged glacier length based on years_combined and locations_combined:
bin_edges=np.arange(XX,XX,3) # times of edges of bins to average over
# calculate number of observations per bin:
bin_count, bin_edges, binnumber \
    = stats.binned_statistic(XX,XX, statistic='count',bins=bin_edges)
# calculate bin averages:
bin_means, bin_edges, binnumber \
    = stats.binned_statistic(XX,XX, statistic='mean',bins=bin_edges)
years_axis=(bin_edges[0:-1]+bin_edges[1:])*0.5

# plot bin-averages:
fig=plt.figure(figsize=(7,2.5),dpi=300)
plt.subplot(1,2,1)
plt.plot(years_axis,bin_means/1000,lw=1,color="b")
plt.xlabel(XX)
plt.ylabel(XX)
plt.xlim(XX,XX)

```

```
# plot number of observations per bin:
plt.subplot(1,2,2)
plt.semilogy(years_axis, bin_count)
plt.xlabel(XX)
plt.ylabel(XX)
plt.xlim(XX)

plt.tight_layout()
plt.show()
```

2. Temperature and glacier length:

(a) Plot glacier length over time for the Morteratsch Glacier (Vadret da Morteratsch) in Switzerland.

```
[ ]: # glacier location time series:
# -----
glacier_years=np.asarray(front_variation['MORTERATSCH, VADRET DA/CH'][0])
glacier_locations=np.asarray(front_variation['MORTERATSCH, VADRET DA/CH'][1])

# first, plot glacier length record
plt.figure(dpi=300)
plt.plot(XX)
plt.ylabel("XX")
plt.xlabel("XX")
```

(b) Plot the annually and globally averaged surface temperature.

(c) Calculate and superimpose the temperature reconstructed from the length record of the above glacier using equation (11.1): Find, via trial and error, a glacier sensitivity c and a response time τ that lead to a temperature that is qualitatively similar to the globally averaged one, with the “hiatus” periods and accelerated warming in the second half of the 20th century.

(d) Superimpose a line plot of the temperature that is in equilibrium with the glacier length at any given time using the sensitivity calculated above. Discuss the character and reasons for the difference between this equilibrium temperature and the one reconstructed using equation (11.1).

```
[ ]: # smooth time series:
# -----
# smooth annual mean time series: use a window size of 7 and a polynomial order of 1
#   ↪ (linear fit)
glacier_locations = savgol_filter(XX,XX,XX)

# use Derleman's formula to get temperature from glacier extent:
# -----
dt=(glacier_years[1:]-glacier_years[:-1])
dLdt=(XX-XX)/dt
c_equilib=3000
tau=20
T_glacier=XX
T_glacier_equilib=XX
```

```
# ----
# plot:
# ----
fig=plt.figure(figsize=(5,6),dpi=300)

plt.plot(XX,label="observed T")
plt.plot(XX,label="reconstructed T")
plt.plot(XX,label="equilibrium")
plt.ylabel("Temperature")
plt.xlabel("year")
plt.legend();
```

3. Idealized glacier length adjustment scenarios:

(a) Plot time series of idealized glacier length adjustment scenarios with a temperature that changes abruptly at $t=0$ and then remains constant, and using the exponential solution to equation (11.1). Use the same c and parameters calculated in question 2. For the first scenario, let the initially imposed temperature anomaly T'_0 be -1 °C and the initial length L'_0 be 4000 m. For the second scenario, $T'_0 = -2$ °C and $L'_0 = 1000$ m.

[]: *# set time array and various parameters for both scenarios:*

```
t=np.asarray(np.arange(0,100,1))
tau=XX
c=XX
T0a=XX
T0b=XX
L0a=XX
L0b=XX

# The solutions for both scenarios:
La=XX
Lb=XX
label_a="$T_0="+repr(T0a)+" , \ L_0="+repr(L0a)
label_b="$T_0="+repr(T0b)+" , \ L_0="+repr(L0b)

# plot both solutions:
fig=plt.figure(figsize=(5,3),dpi=300)
plt.plot(t,La/1000,color="r",label=label_a)
plt.plot(t,Lb/1000,color="b",label=label_b)
plt.xlabel("Time (years)")
plt.ylabel("Lengthn anomaly, $L'(t)$ (km)")
plt.xlim(-1,100)
plt.grid(lw=0.25)
plt.legend()
plt.tight_layout();
```

(b) Optional extra credit: Glacier length response to a varying temperature: Find and plot the solution to equation (11.1) for a linearly changing temperature. Use start and end anomaly temperatures of $T_0 = -0.25$ K, $T_f = 1$ K, climate sensitivity of $c = 3000$ m/K, adjustment time of $\tau = 20$ yr, a total period of 140 yr, and an initial length anomaly of $L_0 = 6000$ m. Explain the structure of the solution you find. Solution: The equation is

$$XX \quad (1)$$

The general solution is

$$L(t) = XX \quad (2)$$

so that,

$$L(t) = XX \quad (3)$$

Now use an initial condition of $L(t = 0) = L_0 = 1.3$ km to find any integration constants, so that the final solution is,

$$XX \quad (4)$$

```
[ ]: # set parameters:
L0=XX # m
c=XX # m/K
tau=XX # years
T0=XX # K
Tf=XX # K
Deltat=XX # total period to calculate/plot, in years
time=np.arange(0,Deltat,Deltat/100)
T=XX # an array with temperature at all times:
L=XX # solution

# plot prescribed linear temperature:
plt.figure(figsize=(7,3),dpi=200)
plt.subplot(1,2,1)
XX

# plot resulting glacier length anomaly record:
plt.subplot(1,2,2)
XX

plt.tight_layout()
print(f"initial L={L[0]:g}")
```

4) Isotopic records from Quelccaya ice cores:

a) Plot the two provided individual ice cores from 1976 and 2016 and rationalize the amplitude of the seasonal cycle seen in them.

b) Plot the decadal bin average of the longer isotopic record for the past 2000 years or so, add shading representing the std of each decadal average. Discuss the seen trends seen since 1750 and in the last few decades in the context of the longer observed record.

```
[ ]: # calculate bin-averaged glacier length:
bin_edges=np.arange(200,2020,10)
bin_means, bin_edges, binnumber \
    = stats.binned_statistic(Quelccaya_years,Quelccaya_delta180 \
                             , statistic='mean',bins=bin_edges)

# bin-averaged std:
bin_std, bin_edges, binnumber \
    = XX

# number of data points per bin:
bin_count, bin_edges, binnumber \
```

```

        = XX
years_axis=(bin_edges[0:-1]+bin_edges[1:])*0.5

# mean of decadal averages:
mean=np.nanmean(bin_means)

# plot:
fig=plt.figure(figsize=(7,3.5),dpi=200)

plt.subplot2grid((1,3), (0, 0), colspan=1)
plt.plot(XX,label="1976")
plt.plot(XX,label="2016")
plt.ylabel('Depth (m)')
plt.xlabel('$\delta^{18}O$ (%)')
plt.legend()
plt.title("XX")

plt.subplot2grid((1,3), (0, 1), colspan=2)
plt.fill_between(XX,color="cyan")
plt.plot(years_axis,bin_means)
plt.xlabel("Year")
plt.ylabel(r'$\delta^{18}O$ (%)')
plt.title("XX")

plt.tight_layout()

```

5) Optional extra credit: The global picture: Calculate a linear regression for each of the given glacier records from the global data set. Draw a global map with circles indicating the latitude and longitude of glaciers, with red indicating negative length trends (slope). Make the size of the circles reflect the magnitude of the trend. Superimpose blue symbols at the location of glaciers showing positive trends.

```

[ ]: # calculate linear regression for each glacier:
slopes = []
for glacier_name in front_variation.keys():
    years=XX
    locations=XX
    # calculate linear regression and store result in dictionary:
    fit=linregress(years, locations)
    slope=XX
    intercept=XX;
    slopes.append(slope)
    # add the regression result to the dictionary:
    if len(front_variation[glacier_name])<4:
        front_variation[glacier_name].append([intercept,slope])
    else:
        front_variation[glacier_name][3]=[intercept,slope]

# calculate standard deviation for normalizing symbol size based on trend magnitude:
std_slope=XX

```

```

# replace trend with normalized one for plotting purposes:
for glacier_name in front_variation.keys():
    normalized_slope=XX
    front_variation[glacier_name][3][1]=normalized_slope

# mark glacier locations on a map, indicating separately those with
# positive and negative trends:

# draw a map with coastlines first:
projection=ccrs.PlateCarree(central_longitude=0.0);
fig,axes=plt.subplots(1,1,figsize=(6,4.5),dpi=300\
    ,subplot_kw={'projection': projection});
axes.set_extent([-180, 180, -90, 90], crs=ccrs.PlateCarree())
axes.coastlines(resolution='110m',lw=0.3,color="grey")
axes.gridlines(linewidth=0.2)

# mark a red/blue markes at glacier locations depending if they are
# decreasing/ increasing based on the linear regression:
# negative trends
N_negative_trends=0
for glacier_name in front_variation.keys():
    lon,lat=front_variation[glacier_name][2]
    if front_variation[glacier_name][3][1]<0:
        N_negative_trends=N_negative_trends+1
        axes.plot(lon,lat,'o',color='r',markerfacecolor='none'\
            ,markeredgewidth=0.5 \
            ,markersize=-3*front_variation[glacier_name][3][1]
            ,transform=ccrs.PlateCarree())

# positive trends:
XX

plt.show()

print(
    f"found {N_negative_trends:d} negative trends and "
    f"{N_positive_trends:d} positive trends."
)

```

(6a) Plot Northern Hemisphere snow area time series for each month.

Add a least-squares linear fit and the corresponding R^2 . Discuss the results.

```

[ ]: fig, axes = plt.subplots(4, 3, figsize=(15, 15), sharey=False, dpi=300)
    axes = axes.flatten()

    start_date = pd.Timestamp('1980-01-01')
    end_date = pd.Timestamp('2024-12-31')
    view_start = pd.Timestamp('1980-01-01')
    view_end = pd.Timestamp('2025-01-01')

    months = list(XX.keys())

```



```

for i, month in enumerate(months):
    ax = axes[i]

    # extract data from dictionary:
    time = SCA_NH_monthly[XX]["XX"]
    area = SCA_NH_monthly[XX]["XX"]

    # Create Decimal Year for Regression (e.g., 2000.5)
    decimal_year = time.dt.year + time.dt.dayofyear / 365.25

    # Linear Regression
    slope, intercept, r_value, p_value, std_err = stats.linregress(XX,XX)

    # Generate the regression line
    line = XX * decimal_year + XX

    # Plotting
    ax.plot(XX, XX, label='Snow Extent', color='steelblue', linewidth=1.5)
    ax.plot(XX, XX, label=f'Trend: ${slope*10:.2f}$M km$^2$/dec; ; $R^2=${r_value**2:.2f}$', color='red', linestyle='--')

    # Formatting
    ax.set_title(XX)
    ax.set_ylabel('Snow extent ($10^6$ km$^2$)')
    ax.set_xlabel('XX')
    ax.legend(loc='best')

plt.tight_layout()

```

Discussion: XX

```

[ ]: ## (6b) ** extra credit: Plot Northern Hemisphere snow area time series for each
    ↪ 3-month season: DJF, MAM, etc.

[ ]: XX code here

[ ]: ## (6c) Plot maps of NH snow cover area during March for the first and last decades of
    ↪ observations, and the difference between the two. Repeat for October.

[ ]: field_first_decade=XX
    field_last_decade=XX
    difference = XX - XX

    longitude_edges = np.append(SCA_longitude - 0.25, SCA_longitude[-1] + 0.25)
    latitude_edges = np.append(SCA_latitude + 0.25, SCA_latitude[-1] - 0.25)

    fig, axes = plt.subplots(1, 3, figsize=(11, 4.2),
                             subplot_kw={"projection": ccrs.NorthPolarStereo()})

    # plot first decade, last, and difference:
    pixels = axes[0].pcolormesh(longitude_edges, latitude_edges, XX, cmap=cmap,

```

```

        shading="flat", antialiased=False,
        rasterized=True, transform=ccrs.PlateCarree())
angle = np.linspace(0, 2 * np.pi, 200)
circle = mpath.Path(np.column_stack((np.sin(angle), np.cos(angle))) * 0.5 + 0.5)
ax.set_extent([-180, 180, 0, 90], crs=ccrs.PlateCarree())
ax.set_boundary(circle, transform=ax.transAxes)
ax.coastlines(resolution="50m", linewidth=0.5, color="0.25")
ax.gridlines(crs=ccrs.PlateCarree(), xlocs=np.arange(-180, 181, 45),
            ylocs=np.arange(0, 91, 30), linewidth=0.4, color="0.45", linestyle=":")

XX

XX

fig.colorbar(pixels, ax=axes[XX], orientation="horizontal",
            extend=extend, label=label)

fig.tight_layout()
plt.show()

```

[]: